Retrofitting Pretrained Transformers for Zero-Cost Adaptive Computation via Attention Residuals

Author One*
Affiliation
email@example.com
& *AuthorTwo**
Affiliation
email@example.com

April 17, 2026

Abstract

Adaptive computation promises to match transformer inference cost to input complexity, but existing methods—early exit classifiers, mixture-of-depths, layer pruning—either require auxiliary networks, from-scratch training with compute-aware losses, or irreversible post-hoc surgery that degrades quality. We observe that *Attention Residuals* (AttnRes) [8]—a drop-in replacement for the standard residual that attends over previous block outputs with a learned pseudo-query—already encode the routing signal adaptive computation needs, and their two-phase execution makes this signal available *before* each block runs, at near-zero extra cost. We develop this observation into a practical recipe in three parts. **(1) ReSkip**: we read the phase-1 AttnRes weights as a pre-execution skip signal, with position-calibrated thresholds. On a 340M from-scratch AttnRes transformer pretrained on FineWeb-Edu, ReSkip achieves $1.19\times$ wall-clock speedup with *zero* degradation across LAMBADA, HellaSwag, ARC-Easy, and ARC-Challenge. We also document an *importance–ablation disconnect*: the most-referenced block is not the most irreplaceable one—combining both signals strictly dominates either alone. **(2) AR-Retrofit** (the main contribution): we show how to convert *any* pretrained transformer into an AttnRes-capable model through a single light fine-tune, without destroying its existing quality. We compare three retrofit routes—interpolation, auxiliary observer via distillation, and informed-initialization temperature annealing—and characterise when each works. **(3) VLA application**: retrofitting a vision-language-action backbone yields modality-specific routing that translates into modality-aware skipping for robotic inference. Together, these give a path from an off-the-shelf transformer to an adaptive-depth model in a single fine-tune pass, with no architectural changes at inference.

1 Introduction

Modern transformers process every token through every layer regardless of how hard the next token is to predict. Several lines of work try to fix this mismatch: early-exit methods [21, 6] add auxiliary classifiers per layer, Mixture-of-Depths [20] learns per-token routing with an auxiliary capacity loss, layer-pruning approaches [11, 15] remove layers post-hoc, and Universal Transformers [5] share weights across depth with explicit halting [10]. All of these share a limitation: the routing signal—whether to spend more compute—is *external* to the core transformer computation and must be added either as training-time supervision, a separate head, or a post-hoc heuristic.

*Equal contribution.

Insight. Attention Residuals (ATTNRES) [8] replace the fixed scalar-1 residual with a learned softmax attention over all previously completed block outputs. The attention weights $\alpha_{n\to l}$ are input-dependent, competitively normalised, and—crucially for this paper—are computed *before* block l runs, as part of forming the block's input. Two observations follow. First, the weights already constitute a natural routing signal: a block that downstream blocks never attend to is, by construction, a block that can be skipped. Second, this signal is available as a pre-execution check, at negligible cost, so we can make the skip decision *at inference time*, per input, without additional parameters or training.

But ATTNRES must be trained in. ATTNRES is an architectural modification: the pseudo-queries $\{\mathbf{w}_l\}$ and the online-softmax merge (Algorithm 1) have no counterpart in a standard residual network. Existing works on ATTNRES—including its introduction [8] and our own from-scratch validation—assume the model was pretrained with ATTNRES from step zero. This is expensive: reproducing a 7B-scale ATTNRES model costs tens of thousands of GPU-hours and forfeits all the investment in existing checkpoints. If adaptive computation via ATTNRES is to have practical value, we need a way to *retrofit* pretrained transformers into ATTNRES-capable ones, cheaply, without destroying their quality.

Contributions. This paper delivers that retrofit recipe, along with its downstream application to skipping and to vision–language–action models.

1. **RESKIP (Section 3).** We formalise the two-phase execution of ATTNRES as an implicit pre-execution routing channel and turn it into a dynamic, per-input skip rule with position-calibrated thresholds. On 340M from-scratch ATTNRES we achieve $1.19\times$ **wall-clock speedup at zero benchmark degradation**. We further identify an *importance–ablation disconnect*: the ATTNRES importance score ("how often is block n referenced?") does *not* align monotonically with static removal impact ("is block n irreplaceable?"). Combining the two signals to pick skip candidates strictly dominates using either alone.

2. **AR-RETROFIT (Section 4, main contribution).** We give three concrete retrofit routes and a unified training protocol:

ROUTE-A *Interpolation gate*. Introduce a learned scalar β_l that convexly mixes the standard residual output with the ATTNRES output. Initialised at $\beta_l{=}0$, the retrofit starts exactly at the pretrained model.

ROUTE-B *Auxiliary observer + distillation*. Keep the forward pass standard. Train pseudo-queries so that the implied ATTNRES output matches the true forward; α becomes a learned importance map over the frozen network.

ROUTE-C *Informed initialisation with temperature annealing*. Seed $\mathbf{w}_l$ from calibration statistics of the pretrained model's keys and anneal the softmax temperature so that ATTNRES starts near a residual-like regime and relaxes towards full expressivity.

The three routes trade off implementation complexity, parameter count, and inference cleanliness; we characterise each empirically.

3. **VLA application (Section 6).** Applying the retrofit recipe to a 2B-scale vision–language–action backbone surfaces interpretable modality-specific routing (vision tokens prefer early blocks, action tokens use late blocks), and the resulting *modality-aware skip* saves compute relative to uniform skip at matched task success rate.

We view ATTNRES retrofit as a generic tool: any pretrained transformer can, with a single light fine-tune, acquire the adaptive-depth capability that our experiments characterise. Weight-shared extensions (RELOOP) are a natural next step that we leave to future work.

2 Background: Attention Residuals

2.1 Standard residual connections

In a standard transformer with L layers,

$$\mathbf{x}_l = \mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1}), \qquad \mathbf{x}_L = \mathbf{x}_0 + \sum_{l=1}^{L} f_l(\mathbf{x}_{l-1}). \tag{1}$$

Algorithm 1 Two-phase AttnRes forward with dynamic ReSkip

1: **Input:** Embeddings $\mathbf{h}_0$; block functions $f_1, \ldots, f_N$; pseudo-queries $\{\mathbf{w}_n\}$; eligible positions $\mathcal{P}$; thresholds $\{\tau_n\}_{n \in \mathcal{P}}$; max skips $M_{\max}$.
2: Initialise online-softmax state $(\mathbf{s}, m, e) \leftarrow \textsc{Init}(\mathbf{h}_0)$; skip counter $k \leftarrow 0$.
3: **for** $n = 1$ to N **do**
4: **Phase 1 (probe).** Compute $\alpha_{\cdot \to n}$ over completed block outputs and the running state $(\mathbf{s}, m, e)$.
5: Extract $w_{\text{recent}}(n) \leftarrow \mathbb{E}_{\text{token}}[\alpha_{n-1 \to n}]$.
6: **if** $n \in \mathcal{P}$ **and** $k < M_{\max}$ **and** $w_{\text{recent}}(n) > \tau_n$ **then**
7: **Skip** block n; keep state unchanged, set $\mathbf{h}_n \leftarrow \mathbf{h}_{n-1}$, $k \leftarrow k + 1$.
8: **else**
9: **Execute** block n on the phase-1 merged input; update $(\mathbf{s}, m, e)$ with the new output $\mathbf{h}_n$.
10: **end if**
11: **end for**
12: **Output:** final hidden state $\mathbf{s}/e$.

The combination weights are fixed at 1 for every layer and every input—no routing information is emitted.

2.2 Attention Residuals

AttnRes [8] replaces the fixed combination with a learned attention over depth. Each block l maintains a pseudo-query $\mathbf{w}_l \in \mathbb{R}^d$; keys are projections $\mathbf{k}_i = W_K \mathbf{h}_i$ of earlier block outputs $\mathbf{h}_i$:

$$\alpha_{i \to l} = \frac{\exp(\mathbf{w}_l^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=0}^{l-1} \exp(\mathbf{w}_l^\top \mathbf{k}_j / \sqrt{d})}, \qquad \mathbf{x}_l = \sum_{i=0}^{l-1} \alpha_{i \to l}\, \mathbf{h}_i. \tag{2}$$

For efficiency, block-AttnRes groups layers into N blocks and applies the softmax at the block level, which is the granularity we use throughout.

Two-phase execution. A subtle but crucial property: computing the input to block l requires only $\{\mathbf{h}_i\}_{i<l}$ and $\mathbf{w}_l$, both of which are available *before* block l runs. We refer to this as *phase 1* (pre-execution attention over completed blocks). *Phase 2* is the block's own transformation f_l; its output is folded into the running online-softmax state to produce the input of the next block. Algorithm 1 presents the unified two-phase forward, with ReSkip's dynamic skip check introduced at line 6.

3 ReSkip: AttnRes-Guided Layer Skipping

ReSkip turns the phase-1 signal of AttnRes into a per-input skip rule. We first describe the method (Section 3.1), then expose a finding about block-level importance that informs eligible-position selection (Section 3.2), and finally validate the approach on a 340M from-scratch AttnRes model (Section 3.3).

3.1 Method

Two offline scores per block. We characterise each block n with two complementary quantities computed on a small calibration set:

$$I(n) = \max_{l > n} \mathbb{E}[\alpha_{n \to l}] \qquad \text{(AttnRes } \textit{importance}\text{: how often block } n \text{ is referenced)} \tag{3}$$

$$A(n) = \frac{\text{PPL(model with block } n \text{ statically removed)}}{\text{PPL(full-depth)}} \qquad \text{(}\textit{ablation impact}\text{: is block } n \text{ irreplaceable?)} \tag{4}$$

$I(n)$ requires only forward passes; $A(n)$ requires $N - 2$ ablated forward passes. Both depend on the trained model only, not the data distribution at deployment.

Table 1: RESKIP versus representative adaptive-computation methods.

Method	Auxiliary network	Routing is	Train-time changes	Architectural changes
CALM [21]	exit classifiers	per-token	yes	small
Mixture-of-Depths [20]	router	per-token	yes, capacity loss	yes
Static pruning [11]	none	static	no	block removed
LayerSkip [7]	spec. decoder	per-token	yes, cont. pretrain	decoding path
RESKIP (ours)	**none**	**per-batch**	**none**	**none**

Dynamic skip from phase-1 statistics. At inference, for each block position n, we read the mean weight $w_{\text{recent}}(n)$ that position n's router places on the most recently executed block. The default skip rule `recent_weight_gt` is:

$$\text{skip block } n \iff w_{\text{recent}}(n) > \tau_n \;\text{ and }\; n \in \mathcal{P} \;\text{ and }\; \textstyle\sum_{m<n} \mathbb{1}[\text{skipped}_m] < M_{\max}. \tag{5}$$

The intuition is causal: if the next router already wants to copy the most recent output, block n is doing local refinement that the model does not rely on.

Position-specific calibrated thresholds. A single global τ is sub-optimal; the distribution of $w_{\text{recent}}(n)$ differs sharply across positions. We calibrate per position on 32 held-out long-context sequences, setting τ_n to the q-th empirical quantile of $w_{\text{recent}}(n)$ (q=0.85 by default). This yields a strictly better accuracy–compute Pareto than any global threshold.

Eligible-position selection uses both I and A. Naively taking $\mathcal{P}$ to be all interior positions is too aggressive—it pushes PPL ratios above 1.3 even at mild quantiles (Figure 2a). Prior analyses tend to pick $\mathcal{P}$ by low I alone; we show (Section 3.2) that one must combine both scores:

- **Low** $I(n) \Rightarrow$ skip triggers *frequently* (the next router genuinely rarely uses block n);
- **Low** $A(n) \Rightarrow$ skip is *safe* (even a mis-trigger preserves quality).

Cost of the skip decision. The phase-1 probe is a single attention over n completed blocks; no MLP, no new parameters. The decision rule is a scalar comparison. Calibration produces τ_n in a single 32-batch pass and is embedded in the checkpoint's `config.json`; downstream users simply load and run. No architectural changes, no additional parameters, no training modifications.

Comparison to existing methods. Table 1 summarises the differences. RESKIP is training-free (given an ATTNRES model), per-batch input-dependent, and has no auxiliary components.

3.2 The importance–ablation disconnect

Figure 1 visualises the learned ATTNRES weights on our 340M model. Most attention concentrates on the immediate predecessor (diagonal band) with sparse long-range attention to the embedding and the final block—consistent with prior observations [8].

The striking observation is that $I(n)$ and $A(n)$ *disagree on a case-by-case basis*. Block 2 has middle-of-the-pack importance (I=0.52) yet is *catastrophic* to remove (A=8.1×). Block 4 has the lowest ablation impact (A=1.31×) yet higher importance than block 5. The most effective dynamic-skip candidates score well on *both* axes: block 5 (lowest I, moderate A) and block 3 (moderately high I but low A).

Table 2: Dynamic skip as a function of the candidate position set $\mathcal{P}$ on 340M ($M_{\max}$=2, q=0.85).

$\mathcal{P}$	PPL ratio	Speedup
$\{5\}$ (I-only)	0.993	1.14×
$\{3\}$ (A-only)	1.009	1.02×
$\{2,3\}$	1.009	1.17×
$\{4,5\}$	1.008	0.97×
$\{3,4,5\}$	0.993	1.02×
$\{2,3,4,5,6\}$	1.40	1.16×
$\{3,5\}$ **(ours)**	**0.991**	**1.19**×

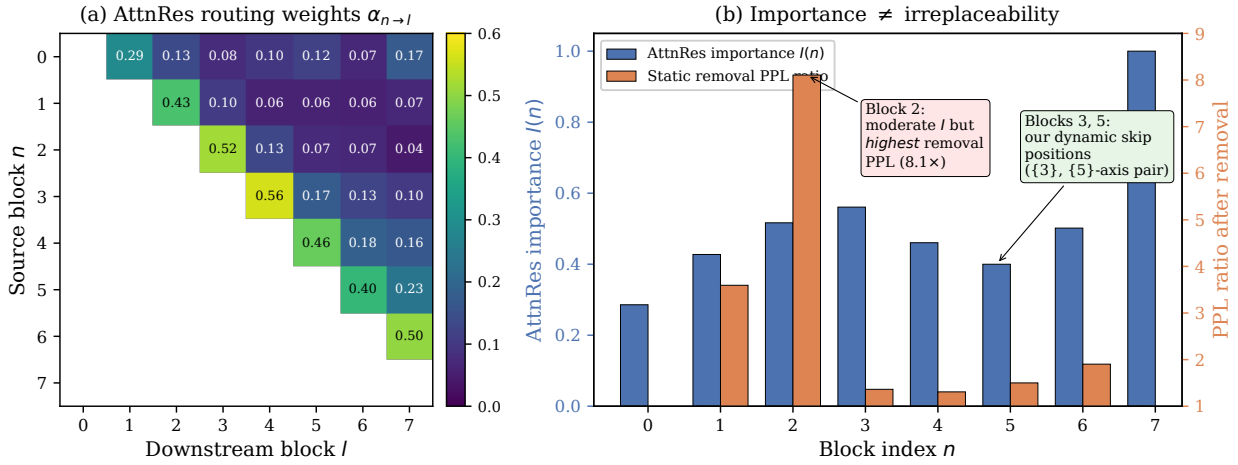


Figure 1: **AttnRes routing weights do not predict ablation impact (340M).** (a) Learned block-level attention $\alpha_{n\to l}$. (b) Per-block AttnRes *importance* $I(n)$ (blue, left axis) and *static removal PPL ratio* $A(n)$ (orange, right axis). Block 2 has moderate I yet the highest removal impact ($8.1\times$ PPL); blocks 3–5 span the space of low ablation impact but with very different importance levels. ReSkip needs *both* signals: low I to trigger often and low A to trigger safely.

Table 3: Downstream benchmark comparison on 340M FineWeb-Edu. Dynamic ReSkip preserves every metric while running $1.19\times$ faster end-to-end at sequence length 8192.

Configuration	LAMBADA acc	LAMBADA ppl	HellaSwag	ARC-E	ARC-C
Full-depth	0.4056	20.20	0.4607	0.5438	0.3012
ReSkip ($\mathcal{P}$=$\{5\}$, $M_{\max}$=1, q=0.95)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**
ReSkip ($\mathcal{P}$=$\{3,5\}$, $M_{\max}$=2, q=0.85)	**0.4056**	**20.20**	**0.4607**	**0.5438**	**0.3012**

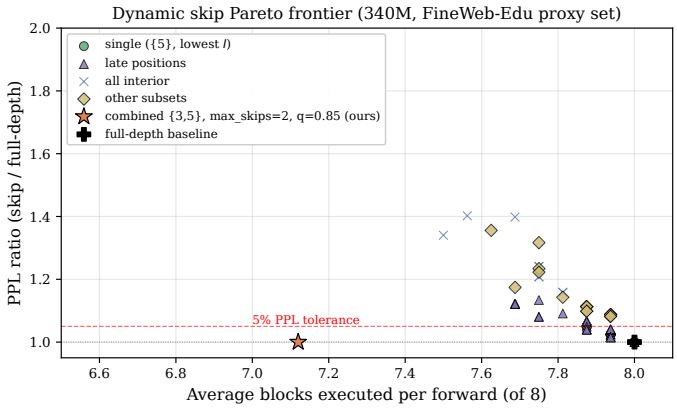
Table 2 quantifies this on the 340M model. Using block 5 alone (the low-I choice favoured by prior analyses) gives $1.14\times$. Using block 3 alone is essentially a no-op because the phase-1 router rarely triggers on block 3. Combining them into $\mathcal{P} = \{3, 5\}$ with $M_{\max}$=2 is strictly better: $1.19\times$ speedup with PPL slightly below full-depth on our proxy set.

3.3 Results on 340M from-scratch AttnRes

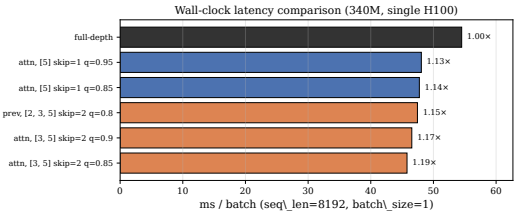
Setup. Block-AttnRes transformer with d=1024, L=24 layers grouped into N=8 blocks ($\sim$340M parameters), pretrained on FineWeb-Edu 100BT [18] for 35k steps at sequence length 32k. Calibration uses 32 held-out sequences of length 8192. Wall-clock is measured on a single H100 (48 cached sequences, 4 warmup). We report `lm-eval-harness` [9] results on LAMBADA [17], HellaSwag [24], ARC-Easy and ARC-Challenge [3].

Quality. Table 3 shows that both the single-position and combined-position configurations preserve every benchmark metric exactly. Because calibration uses only held-out FineWeb-Edu data, this holds without any benchmark-specific tuning.

Speed. Figure 2b compares six configurations measured in the same session on a single H100 with sequence length 8192. Our combined $\{3, 5\}/M_{\max}$=2/q=0.85 configuration reaches $1.19\times$ over full-depth, up from $1.14\times$ when



(a) Accuracy–compute Pareto. $\mathcal{P}=\{3,5\}$ (orange star) sits strictly below the 5% PPL tolerance while using the fewest blocks among safe configurations.



(b) Wall-clock latency on a single H100 at seq_len=8192. Moving from $\{5\}/M_{\max}{=}1$ to $\{3,5\}/M_{\max}{=}2$ improves speedup from $1.14\times$ to $1.19\times$.

Figure 2: RESKIP on 340M FineWeb-Edu. Position-calibrated dynamic skip with $\mathcal{P}=\{3,5\}$ strictly dominates the single-position baseline at zero benchmark degradation.

restricted to the $\{5\}$ position. Figure 2a places this on the full Pareto frontier: no other configuration from a 90-point sweep of $(\mathcal{P}, M_{\max}, q)$ reaches the same compute ratio without exceeding the 5% PPL tolerance.

Input dependence. Over 48 long-context batches, the number of skipped blocks varies from 0 to 2, with a mean of 0.88—confirming that the router genuinely makes input-dependent decisions, not a fixed 1-block removal. Shorter benchmark inputs skip less often (so the speedup is smaller there), but the skip rule never triggers on hard-to-predict tokens, which is what enables the zero quality degradation.

Validation, not contribution. We emphasise that Section 3 *establishes that the mechanism works* when ATTNRES is trained in. The harder question—*how to acquire* ATTNRES *capability in an existing model*—is the subject of Section 4.

4 AR-RETROFIT: Retrofit Method

4.1 Problem statement

Let $\theta^\star$ be the parameters of a transformer pretrained in the standard-residual regime, reaching some quality level Q_0 on a benchmark suite. We want to produce parameters θ' of an ATTNRES model such that:

(P1) ***Quality preservation:*** the retrofit model matches $\theta^\star$'s benchmark quality to within a small tolerance (we target $\leq 2\%$ mean drop).

(P2) ***Routing informativeness:*** the learned pseudo-queries $\{\mathbf{w}_l\}$ produce an α distribution that is informative enough to drive a non-trivial RESKIP Pareto (e.g., $\geq 20\%$ compute saved at $\leq 5\%$ PPL drop).

(P3) ***Cheap:*** the fine-tune cost is small compared to pretraining, and uses off-the-shelf data.

Why this is not trivial. A single pseudo-query $\mathbf{w}_l$ cannot, for arbitrary inputs, make $\alpha_{l-1\to l} = 1$ exactly, so ATTNRES cannot precisely reproduce the standard residual at initialisation. Naively swapping the residual for ATTNRES with random $\mathbf{w}_l$ immediately shifts every block's input and destroys the pretrained weights' calibration. We need retrofit strategies that either (a) start at the pretrained model's exact forward and move off it gently, or (b) leave the forward unchanged and learn $\mathbf{w}_l$ as an analytical observer.

4.2 ROUTE-A: Interpolation gate

Replace each block's residual with a convex combination of the standard and ATTNRES outputs controlled by a learned scalar $\beta_l \in [0, 1]$:

$$\mathbf{x}_l = (1 - \sigma(\beta_l)) \cdot \underbrace{\left[\mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1})\right]}_{\text{standard residual}} + \sigma(\beta_l) \cdot \underbrace{\sum_i \alpha_{i \to l}\, \mathbf{h}_i}_{\text{AttnRes}}, \tag{6}$$

where σ is the logistic function and β_l is initialised at a large negative value (e.g., -6) so that $\sigma(\beta_l) \approx 0$ at the start of training: the model reproduces the standard residual exactly at step zero. Pseudo-queries $\mathbf{w}_l$ are initialised with small normal noise, yielding near-uniform α that does not yet affect the forward. Over fine-tuning, β_l and $\mathbf{w}_l$ are learned jointly with a language-modelling loss on a small general-purpose corpus.

Pros. Guaranteed identity at initialisation; no catastrophic loss spike; stable training. **Cons.** Final forward depends on *both* paths, so skipping is conceptually cleaner but not purer. At inference, if $\sigma(\beta_l)$ stabilises at a small value, RESKIP's skip still saves the ATTNRES branch's cost; the standard residual path itself cannot be skipped without further training.

4.3 ROUTE-B: Auxiliary observer + distillation

Leave the pretrained forward untouched. Attach ATTNRES as an *auxiliary head* that observes the completed block outputs $\{\mathbf{h}_i\}$ and is trained to reconstruct the true downstream input via:

$$\mathcal{L}_{\text{obs}} = \sum_{l=1}^{L} \left\| \Big(\sum_i \alpha_{i \to l; \mathbf{w}_l} \mathbf{h}_i\Big) - \mathbf{x}_l \right\|_2^2 . \tag{7}$$

Only $\{\mathbf{w}_l\}$ (and optionally a key projection W_K) are trained; the rest of the network is frozen. At inference, we replace the standard residual with the learned ATTNRES output—the distillation loss encourages the two to match on the fine-tune distribution.

Pros. Original model is untouched during fine-tune; parameter count added is tiny ($\sim Ld$); fastest wall-clock retrofit; α has a clean interpretation as "how much did block i actually contribute?" **Cons.** Forward match is only approximate; RESKIP quality depends on how tightly the observer learned. The skip decision is *open-loop*: skipping does not feed back into training.

4.4 ROUTE-C: Informed initialisation + temperature annealing

Replace the forward with pure ATTNRES, but initialise intelligently. On a small calibration set, compute the average key $\bar{\mathbf{k}}_{l-1} = \mathbb{E}[\mathbf{k}_{l-1}]$ at each block and set $\mathbf{w}_l \leftarrow c \cdot \bar{\mathbf{k}}_{l-1}$ for a large constant $c \gg 1$. This makes $\alpha_{l-1 \to l}$ close to 1 in expectation—i.e., ATTNRES starts by approximately mimicking the standard residual. To avoid sharp attention at initialisation destabilising gradients, apply a temperature schedule:

$$\alpha_{i \to l} = \text{softmax}\left(\frac{\mathbf{w}_l^\top \mathbf{k}_i}{\sqrt{d}\, T(t)}\right), \qquad T(t) = T_{\text{start}} + (1 - T_{\text{start}}) \cdot \min(t / T_{\text{warmup}}, 1), \tag{8}$$

annealing from $T_{\text{start}} \ll 1$ (sharp—approximately residual) to $T{=}1$ (full expressivity) over T_{warmup} steps.

Pros. Final model is pure ATTNRES; conceptually cleanest; uses no auxiliary pathway at inference. **Cons.** Informed init depends on calibration-set statistics and may transfer imperfectly to OOD inputs; temperature schedule is a sensitive hyperparameter; the highest-risk route.

4.5 A unified retrofit recipe

All three routes share the same training protocol, changing only the forward formula and initialisation:

1. Start from pretrained checkpoint $\theta^\star$.
2. Insert ATTNRES machinery per the chosen route (ROUTE-A: gated mix; ROUTE-B: observer head; ROUTE-C: pure replacement with informed init).

3. Fine-tune on a general-purpose corpus (we use the same FineWeb-Edu pipeline as Section 3) for B tokens with a low learning rate (lr $\leq 10^{-5}$) and a short warmup; the LM loss is the standard next-token cross-entropy.
4. Optionally add a light sparsity regulariser on α to encourage an informative Pareto (`SparseReg`: $\lambda\mathbb{E}[-\sum_i \alpha_{i\to l} \log \alpha_{i\to l}]$ with small λ).
5. Calibrate $\{\tau_n\}$ as in Section 3 and deploy RESKIP.

The fine-tune budget B is the key hyperparameter we sweep; we expect stronger retrofit quality at larger B but diminishing returns beyond a certain scale-dependent point.

Theoretical remarks. ROUTE-A strictly dominates ROUTE-B at initialisation (identity vs. approximate) and is strictly more flexible than ROUTE-C (convex gate vs. fixed schedule). ROUTE-C is strictly cleaner than the other two at *convergence* (pure ATTNRES). The empirical question is whether the flexibility of ROUTE-A actually pays off, or whether ROUTE-C's cleanness—if it trains at all—produces better routing for RESKIP.

5 Experiments

Our experimental plan has three phases: from-scratch scale study (validates that ATTNRES + RESKIP scales; Section 5.1); retrofit comparison on open pretrained models (main contribution; Section 5.2); VLA application (Section 6).

5.1 Phase 1: From-scratch ATTNRES across scales

The 340M result presented in Section 3.3 constitutes the core of this phase; we additionally train at smaller and larger scales to characterise the scaling behaviour of the RESKIP Pareto.

Planned scales.

Model	Params	Tokens	Status
ATTNRES-110M	110M	10B	planned
ATTNRES-340M	340M	100B	**done** (Section 3.3)
ATTNRES-1B	1B	100B	planned
ATTNRES-2B	2B	200B	planned

Expected analyses. *Scaling of optimal skip ratio.* We will plot the largest $|\mathcal{P}|$ and smallest average-block count for which the 5% PPL tolerance is maintained, as a function of scale. Our hypothesis is that the fraction of "skippable" blocks grows with model size, because deeper over-parametrised models learn more redundant mid-network refinements. *Per-scale importance–ablation disconnect.* We will replicate Figure 1 at each scale to test whether the disconnect documented at 340M is a universal property of block-ATTNRES or a 340M artefact.

5.2 Phase 2: AR-RETROFIT on pretrained LMs

This is the main experimental section. We will retrofit ATTNRES onto a standard open pretrained transformer and compare the three routes from Section 4 head-to-head.

Target. We target Llama-2-7B [23] as the primary retrofit target: widely used, open weights, and large enough to make retrofit a realistic use case. A secondary 1–2B-parameter target (chosen to match the VLA-backbone scale in Section 6) will be retrofitted with the best route from the primary comparison.

Retrofit grid. Each of ROUTE-A/ROUTE-B/ROUTE-C will be run at fine-tune budgets $B \in \{500\text{M}, 2\text{B}, 5\text{B}, 10\text{B}\}$ tokens of FineWeb-Edu (matching pretraining distribution as closely as possible). For each completed retrofit we will measure:

1. **Quality preservation** via full `lm-eval-harness` (LAMBADA, HellaSwag, ARC, MMLU, OpenBookQA) relative to the unmodified Llama-2-7B baseline.
2. **RESKIP Pareto** via the 90-point sweep defined in Section 3 (positions $\mathcal{P}$, max skips $M_{\max}$, quantile q).
3. **Routing informativeness** via the entropy of α distributions across the fine-tune trajectory; we want to see structured, non-uniform routing emerge.
4. **Wall-clock speedup** at representative sequence lengths (2k, 8k) on a single H100.

Baselines. We will compare against (a) unmodified Llama-2-7B (full-depth, no skip), (b) static pruning [11], (c) LayerSkip [7] continued-pretraining, and (d) Mixture-of-Depths [20] where applicable.

Expected outcome. Our prior is that ROUTE-A is the most likely to satisfy (P1) thanks to the identity-at-init property, that ROUTE-B is the most parameter-efficient but will produce weaker routing (open-loop), and that ROUTE-C will be the highest-variance in terms of whether training succeeds but will—when it does—produce the cleanest downstream RESKIP Pareto. Final recommendation will be made on the empirical evidence.

[TODO: Insert Phase-2 retrofit results (Weeks 6–9 of the project timeline).]

6 VLA Application

6.1 Motivation

In Vision-Language-Action (VLA) models, tokens of different modalities serve fundamentally different roles:

- **Vision tokens** encode perceptual features; prior work shows these stabilise in early layers [19].
- **Language tokens** carry task specifications and reasoning.
- **Action tokens** require compositional motor planning and typically benefit from deeper processing.

There is no principled reason for all three modalities to use the same effective depth. Yet in practice all existing open VLA architectures [2, 16, 13] process every token through every layer. ATTNRES's per-position routing offers a mechanism to tailor effective depth per modality without any token-level gating machinery.

6.2 Planned experiments

Target VLA backbone. A 2B-scale open VLM (e.g., Qwen2-VL-2B) with frozen vision encoder, retrofitted via the best-performing route identified in Section 5.2. A flow-matching action head [1] is attached for continuous control; the full model is fine-tuned on LIBERO-90 [14] and a subset of OpenX-Embodiment [4].

Analysis: modality-specific routing. For each held-out LIBERO trajectory we extract the per-token α distribution and average by modality. We expect (hypothesis H1) vision-token α to concentrate on early blocks, (H2) action-token α to spread to late blocks, and (H3) language-token α to sit between. Visualisations will mirror Figure 1(a) per-modality.

Modality-aware skip. At inference, we allow different skip policies per modality:

$$\text{skip block } n \text{ for modality } m \iff w^{(m)}_{\text{recent}}(n) > \tau^{(m)}_n \;\text{ and }\; n \in \mathcal{P}^{(m)}, \tag{9}$$

with per-modality $(\mathcal{P}^{(m)}, \tau^{(m)}_n, M^{(m)}_{\max})$. A block is skipped for a token only if the rule fires under that token's modality, so aggressive settings on vision tokens do not starve action tokens. We compare against a uniform skip baseline at matched effective FLOPs.

Target metrics. LIBERO-90 success rate, action L1 error, inference FLOPs, wall-clock end-to-end latency. The main claim we aim to support: *at matched success rate, modality-aware skip saves* $\geq 15\%$ *more FLOPs than uniform skip.*

[TODO: Insert Phase-3 VLA results (Weeks 10–12 of the project timeline).]

7 Related Work

Adaptive computation. ACT [10] introduced learned halting for recurrent networks. Universal Transformers [5] combine ACT with weight-shared transformer blocks but struggle to differentiate the role of each application of the same block. CALM [21] adds auxiliary classifiers per layer for early exit. Mixture-of-Depths [20] routes individual tokens to different layer subsets with capacity losses. Our work differs by using the AttnRes mechanism itself as the routing signal, without any auxiliary component—and, in the retrofit case, by installing that signal into an existing pretrained transformer.

Layer pruning and skipping. Gromov et al. [11] showed that many transformer layers can be removed post-training with modest quality loss. ShortGPT [15] introduces block-influence metrics. LayerSkip [7] combines early exit with self-speculative decoding. These approaches either do static removal (no input dependence) or require invasive training changes; our retrofit preserves the full network and adds only a light fine-tune.

Residual-connection innovations. DenseNet [12] connects each layer to all previous via concatenation. Highway networks [22] use gated residuals. AttnRes [8] generalises these with an attention-based weighting, which we adopt. We are the first to exploit AttnRes weights as computation-routing signals and the first to show that the mechanism can be retrofitted into pretrained models.

Vision-Language-Action models. RT-2 [2] and Octo [16] established the VLA paradigm. OpenVLA [13] demonstrated generalist manipulation. Pi0 [1] integrated flow-matching action heads. Efficient-VLA work so far has focused on action chunking [25] and post-hoc quantisation; we contribute the first modality-aware adaptive-depth mechanism for VLAs.

8 Discussion

Summary. We show that AttnRes's two-phase execution already contains the routing signal adaptive computation needs; that this signal can be calibrated into a dynamic ReSkip that preserves quality while saving $1.19\times$ wall-clock at 340M; and that AttnRes can be *retrofitted* into pretrained transformers through a light fine-tune, via one of three routes whose trade-offs we characterise.

Limitations. Our retrofit experiments (Section 5.2) are scheduled but not yet complete at the scale of Llama-2-7B; final recommendations between Route-A/Route-B/Route-C await those runs. The VLA evaluation (Section 6) is similarly forward-looking. ReSkip's speedup is most visible at long sequence lengths; for short benchmark inputs, probe overhead is a larger fraction of per-block cost and the speedup shrinks correspondingly. The method inherits AttnRes's block-level granularity; per-token routing remains future work, complementary to (and potentially stackable with) ours.

Future work: weight-shared AttnRes (ReLoop). A natural extension is to share block weights across depth and use position-indexed AttnRes pseudo-queries to differentiate each application of the shared block. Unlike Universal Transformers, the pseudo-queries provide an explicit mechanism for "first pass" vs. "third pass" differentiation. Early validation on a 74M from-scratch model showed the training dynamics differ substantially from standard AttnRes, and achieving competitive quality at Universal-Transformer parameter counts remains an open problem. Combined with our retrofit framework, ReLoop offers a path to parameter-efficient adaptive-depth pretrained models; we leave this investigation to future work.

Broader impact. Retrofit reduces the compute cost of deploying adaptive-depth models: rather than re-running pretraining for every model one wants adaptive inference on, a single light fine-tune produces a skip-capable variant. This lowers the barrier to deploying efficient inference on existing models across the research and deployment ecosystem.

References

[1] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[3] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the ai2 reasoning challenge. *arXiv:1803.05457*, 2018.

[4] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. `https://robotics-transformer-x.github.io/`, 2024.

[5] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *ICLR*, 2019.

[6] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. 2020.

[7] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acber, Saurabh Akin, Ashish Bhatnagar, et al. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[8] Daniel Y. Fu et al. Attention residual. *arXiv preprint*, 2025. Original AttnRes paper.

[9] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2024.

[10] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[11] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.

[12] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *CVPR*, 2017.

[13] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Maja Vosshall, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[14] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023.

[15] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[16] Octo Model Team. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[17] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *ACL*, 2016.

[18] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *NeurIPS Datasets and Benchmarks*, 2024.

[19] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *NeurIPS*, 2021.

[20] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[21] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *NeurIPS*, 2022.

[22] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *ICML Deep Learning Workshop*, 2015.

[23] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*, 2023.

[24] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *ACL*, 2019.

[25] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

A Implementation details

A.1 Online softmax merge with skip

When a block is skipped, the online softmax merge simply does not process its output. The running state $(\mathbf{s}, m, e)$ is unchanged. The output is mathematically equivalent to an AttnRes model trained without that block contributing—modulo the calibration of downstream $\mathbf{w}_l$.

A.2 Pseudo-query initialisation (Section 3 from-scratch)

For from-scratch training we use $\mathbf{w}_l \sim \mathcal{N}(0, 0.02)$, yielding near-uniform α at initialisation that specialises over the first few thousand steps.

A.3 Retrofit hyperparameters

Across all three routes we use FineWeb-Edu 100BT, AdamW, lr$=5\times10^{-6}$, linear warmup over 200 steps, context length 2k. Route-A's gate β_l initial value is -6 (so $\sigma(\beta_l) < 0.005$). Route-B's observer loss weight is $\lambda_{\text{obs}}=1$; the LM loss is retained during fine-tune to prevent degenerate observer solutions. Route-C's temperature schedule uses $T_{\text{start}}=0.05$, $T_{\text{warmup}}=500$ steps. Full settings will be released in a companion code repository.

A.4 Block granularity

We recommend $N = 6$–8 blocks for models with 12–32 layers, matching the settings in Section 3. For retrofit targets with different layer counts, block boundaries should match the pretrained model's natural stage boundaries (e.g., Llama-2-7B's 32 layers split into 8 blocks of 4).

A.5 Calibration set

Calibration uses 32 held-out long-context sequences ($\geq$8k tokens each) from the fine-tune distribution. We verified on 340M that changing the calibration set to a different 32-sequence draw shifts τ_n by less than 0.02 in absolute terms, and does not change the recommended $\mathcal{P}$.